

The Latent Structure of Dictionaries

Vincent-Lamarre et al. (2014)

情報経営システム工学 4年
ロペスチャパホセ

5月12日2026年

Step 0: Building the graph

What do we want to build?

We want a structure that captures, for every word in words it depends on to be defined, and which other words depend on it.

We want to be able to ask questions like:

1. Is this word used by anyone the dictionary, which other else?
2. How many words need this word in their definition?
3. Are there groups of words that define each other in circles?

How dictionary data actually looks like.

The Longman Dictionary of Contemporary English

```
{
  "happy": {
    "pos": "adjective",
    "definition": "feeling pleased and satisfied about something"
  },
  "pleased": {
    "pos": "adjective",
    "definition": "feeling happy about something that has happened"
  },
  "satisfied": {
    "pos": "adjective",
    "definition": "pleased because you have got what you wanted"
  }
}
```

Step 0: Building the graph

Preprocessing

Step 1 — Tokenization

Split the raw definition string into individual words. Lowercase everything. Punctuation and hyphens are treated as separators.

Step 2 — Stop word removal

Remove function words that carry no definitional meaning — articles, prepositions, conjunctions, pronouns, auxiliaries.

Step 3 — Lemmatization

Reduce every word to its base form so that different conjugations of the same word map to the same node. Why this matters: If "runs," "running," and "ran" are treated as separate nodes, the same concept splits into three disconnected points in the graph. Lemmatization collapses them into one.

Step 4 — Vocabulary filtering

Keep only words that have their own entry in the dictionary. If a definition uses a word with no entry — a proper noun, abbreviation, or rare term — that word is dropped. No node, no edge. This keeps the graph self-contained: every node in the graph can itself be looked up and defined.

Step 5 — First sense only

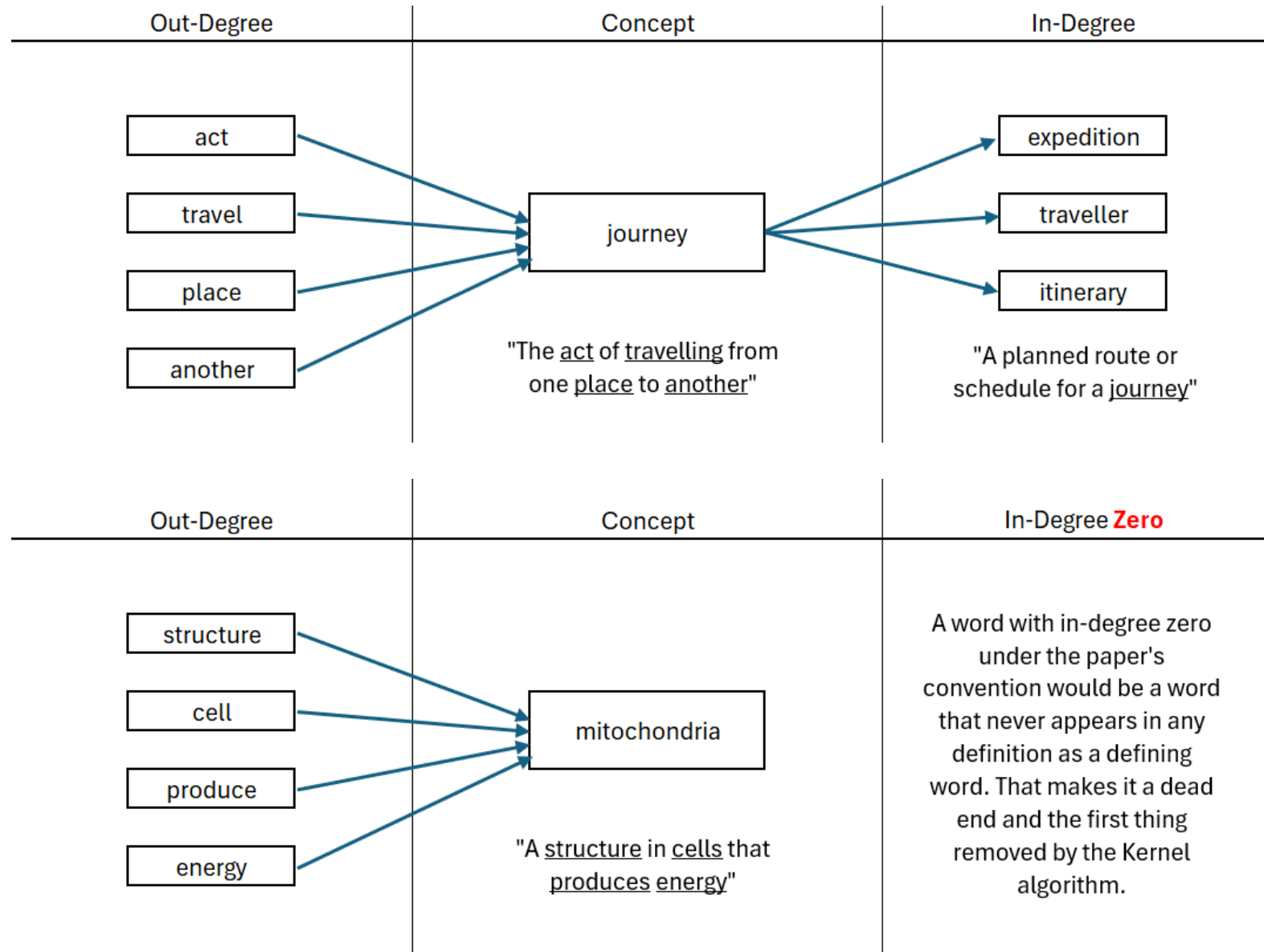
Each word has only one definition processed — the first one listed. "Bank" means financial institution, not riverbank. This makes the graph tractable but loses polysemy. The paper acknowledges this as a limitation.

"A dog is a domesticated four-legged animal" →
["a", "dog", "is", "a", "domesticated", "four-legged", "animal"]

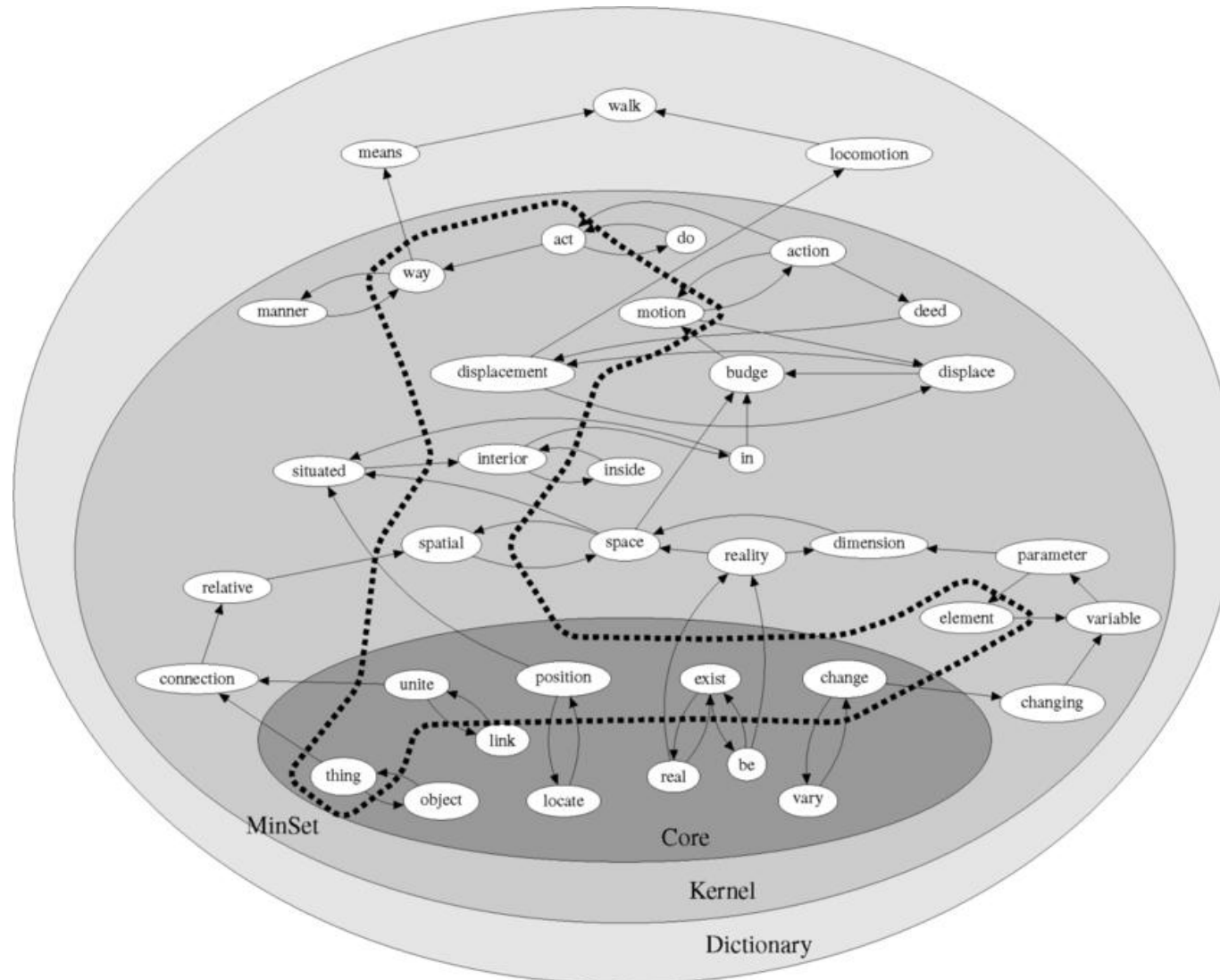
["a", "dog", "is", "a", "domesticated", "four-legged", "animal"] →
["domesticated", "four-legged", "animal"]

Before	After
runs	run
running	run
ran	run

Step 0: Building the graph



Step 0: Building the graph



Step 1: Kernel

What is the kernel?

The Kernel is the minimum vocabulary through which the dictionary defines itself. It tells you which words are part of the dictionary's self-sustaining definitional core. The other 90% of words are peripheral — they enrich the vocabulary but the definitional machinery does not depend on them.

```
function build_kernel(graph):  
  
    changed = true  
  
    while changed is true:  
  
        changed = false  
        words_to_remove = empty list  
  
        for each word in graph:  
            if in_degree(word) == 0:  
                add word to words_to_remove  
                changed = true  
  
        for each word in words_to_remove:  
            remove word from graph  
            remove all arrows connected to word  
  
    return graph    ← this is the Kernel
```

Intuition

Imagine you are trying to keep only the words that matter for the definitional system.

A word that never appears in any other word's definition is useless to the network, it consumes definitions but contributes nothing.

You can remove it without affecting anything else. Strip those words away, and some words that previously had arrows pointing into them from the removed words may now also be useless. Remove those too. Keep going until you cannot remove anything else. What remains is the **Kernel**.

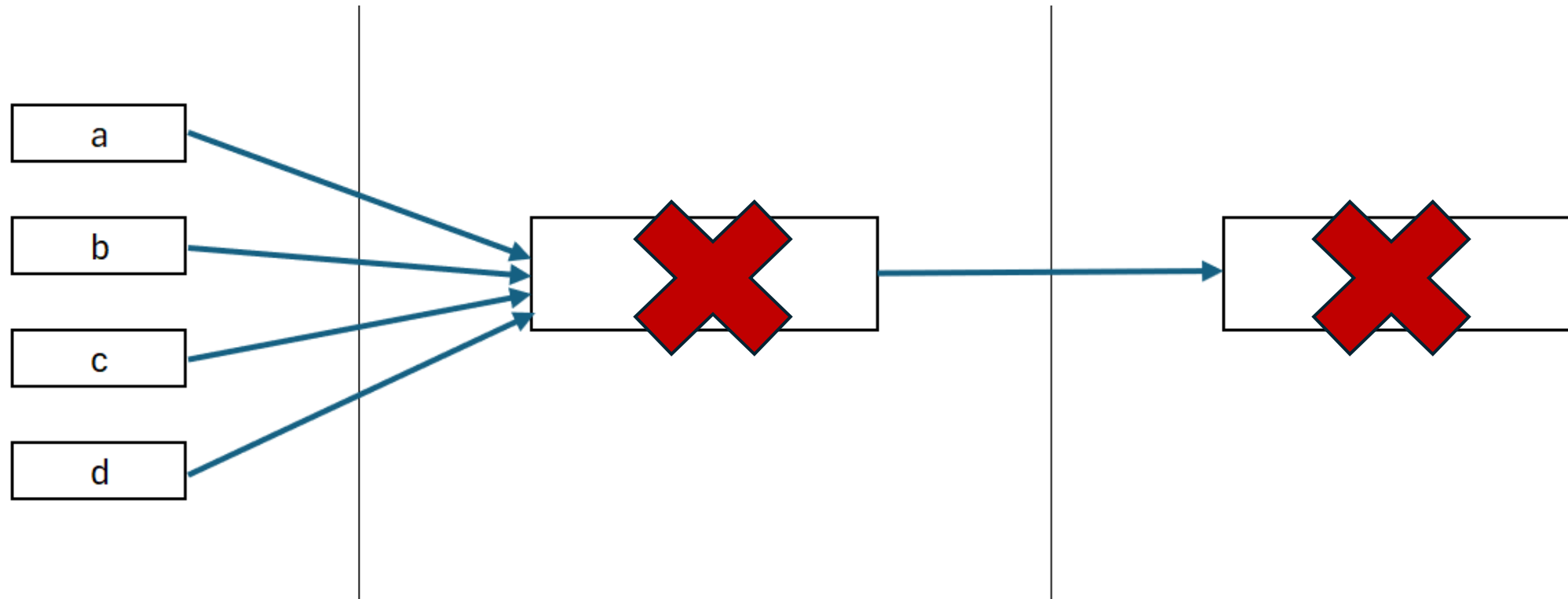
Why the Kernel is unique

No matter what order you remove words, you always end up with the same Kernel.

Removing a word with in-degree zero cannot change whether any other word has in-degree zero except by potentially exposing new zeros. The final stable state is always identical regardless of removal order.

Every dictionary has exactly one Kernel. It's about 10% of the dictionary

Step 1: Kernel



Step 2: Core

What is the Core?

The Core is the giant Strongly Connected Component (SCC) at the center of the Kernel. An **SCC** is a set of nodes where every node is reachable from every other node by following directed edges. After all dead-end words have been stripped away, one group remains where every word can reach every other word by following definition arrows in any direction, from any starting point. That group is the Core. **Around 75% of the Kernel and about 8% of the dictionary.**

Why the Core is special

1. **Self-contained.** No word inside the Core needs anything from outside the Core to be defined.
2. **Mutually reachable.** You can navigate from any word to any other word inside it by following arrows.
3. **Most abstract layer.** Core words tend to be the most frequent, earliest-acquired, and most abstract words in the language — "make," "know," "go," "say," "good," "time," "place."

What does the Core tells you?

The words inside the core sit at the definitional center of everything and corresponds precisely to the words humans learn earliest and use most frequently.

Step 2: Core

To construct the Core, there are two main approaches:

- **Tarjan's** — one pass through the graph. Tracks two numbers per word: discovery time and lowlink. When a word's lowlink equals its own discovery time, an SCC boundary is detected and the group is popped off the stack immediately.
- **Kosaraju's** — two passes. First pass records the order words finish being explored. Second pass reverses all arrows and walks the graph again using that finishing order. Everything reachable in the reversed graph from each starting point is one SCC.

The practical difference

Tarjan needs one traversal and a stack. Kosaraju needs two traversals and a reversed copy of the graph. Tarjan is generally preferred in NLP research because it avoids storing the reversed graph, which matters at dictionary scale with millions of edges.

```
function tarjan_scc(graph):

    index_counter = 0
    stack = empty
    lowlink = {}      ← lowest discovery time reachable
    index = {}        ← discovery time of each node
    on_stack = {}     ← is this node currently on the stack?
    all_sccs = []     ← list of all SCCs found

    function strongconnect(node):

        index[node] = index_counter
        lowlink[node] = index_counter
        index_counter += 1
        stack.push(node)
        on_stack[node] = true

        for each neighbor of node:
            if neighbor not yet visited:
                strongconnect(neighbor)      ← recurse
                lowlink[node] = min(lowlink[node], lowlink[neighbor])
            else if neighbor is on stack:
                lowlink[node] = min(lowlink[node], index[neighbor])

        if lowlink[node] == index[node]:     ← this node is SCC root
            new_scc = empty list
            repeat:
                w = stack.pop()
                on_stack[w] = false
                new_scc.add(w)
            until w == node
            all_sccs.add(new_scc)

    for each node in graph:
        if node not yet visited:
            strongconnect(node)

    return all_sccs
```

```
function kosaraju_scc(graph):

    # — PASS 1 —————
    # DFS on original graph
    # Push each word onto stack when fully explored

    visited = empty set
    stack = empty stack

    function dfs_pass1(word):
        visited.add(word)
        for each neighbor of word:
            if neighbor not in visited:
                dfs_pass1(neighbor)
        stack.push(word)      ← push AFTER all neighbors explored

    for each word in graph:
        if word not in visited:
            dfs_pass1(word)

    # — PASS 2 —————
    # Reverse all arrows in the graph

    reversed_graph = reverse_all_edges(graph)

    # DFS on reversed graph, starting from stack top
    # Everything reachable = one SCC

    visited2 = empty set
    all_sccs = []

    function dfs_pass2(word, current_scc):
        visited2.add(word)
        current_scc.add(word)
        for each neighbor of word in reversed_graph:
            if neighbor not in visited2:
                dfs_pass2(neighbor, current_scc)

    while stack is not empty:
        word = stack.pop()
        if word not in visited2:
            new_scc = empty set
            dfs_pass2(word, new_scc)
            all_sccs.add(new_scc)

    # — RESULT —————
    return max(all_sccs, key=len)      ← largest SCC = Core
```

Step 3: MinSet

What is the MinSet?

The MinSet is the smallest possible set of words that must be learned from real-world experience — not from definitions — for every other word in the dictionary to become learnable by definition alone. **Roughly 1% of the dictionary.**

How it works mathematically

Every cycle needs at least one word removed to be broken. The MinSet is the smallest set of words that, if removed from the graph, eliminates every cycle simultaneously. Once those words are removed — meaning grounded in experience — every remaining word can be reached through definition chains that no longer loop. The entire vocabulary becomes learnable from those grounded foundations.

How are MinSets found?

Two approaches depending on dictionary size:

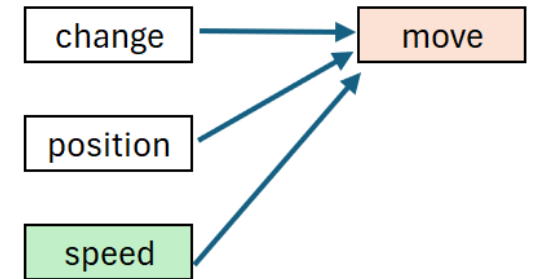
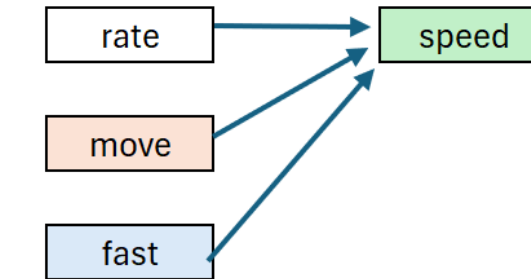
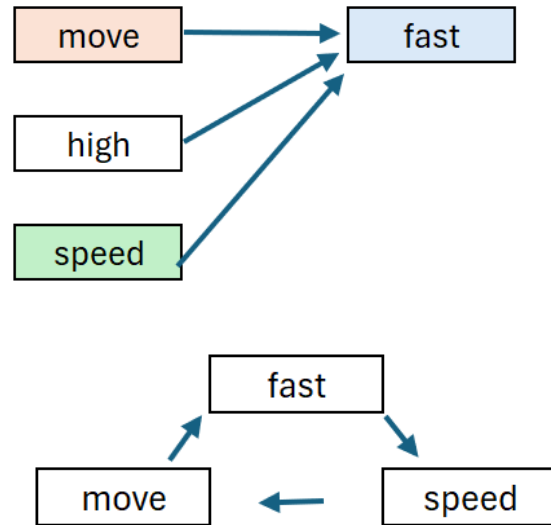
- **Exact — CPLEX (small dictionaries)** A mathematical optimization solver. You give it the variables (one per word, 0 or 1), the objective (minimize total words selected), and the constraints (every cycle must lose at least one word). CPLEX searches the solution space using branch and bound — pruning combinations it can prove will never beat the current best — until it finds the provably optimal MinSet.
- **Approximate — greedy (large dictionaries)** Repeatedly pick the word participating in the most remaining cycles and remove it. Not guaranteed to be the absolute minimum but close enough, and runs in a fraction of the time. Used for Webster (~248,000 words).

Why NP-hard?

The number of cycles grows exponentially with graph size. A dictionary Kernel has potentially millions of overlapping cycles. Finding the smallest set covering all of them simultaneously requires checking an exponential number of combinations in the worst case — no known algorithm guarantees the exact minimum in polynomial time. Even CPLEX, with all its pruning, took days of compute time for the larger dictionaries.

Step 3: MinSet

Fast: Moving at high speed
Speed: The rate at which something moves fast
Moving: Changing position at a speed



Removing "fast" from the graph means it enters the MinSet, it must be learned through direct experience, not definition. Once grounded, the loop breaks. "Speed" can now be learned from its definition using "fast", which we already know, plus its other content words. "Moving" does the same. You pay the experiential cost once for one word, and the definitional system unlocks everything else that was trapped in the cycle with it.